Въпрос номер 21 от конспекта за държавен изпит на специалност «Софтуерно инженерство»

Използване на XML за структуриране, валидация, обработка и представяне на документно съдържание.

Изготвил: Станислава Бобчева, спец. Софтуерно инженерство, 4-ти курс

Съдържание

Добре структуриран XML

Произход на маркъп езиците и основни концепции в XML

Маркъп език (според Уикипедия) е модерна система за анотиране на текст по такъв начин, че маркирането синтактично се различава от самия текст. Терминът произлиза от редактирането на ръкописи, където с друг цвят например се пишат забележките от редакцията. Основните характеристики на маркъп езиците са:

- ✓ За стилистично оформяне – например от HTML <I>, <B>, <U>
- ✓ За структуриране – например от HTML <P>,
, <H2>
- ✓ За добавяне на семантично значение (мета данни) – например от HTML <TITLE><META NAME=keywords CONTENT="….">
- ✓ За добавяне на функционалност – например от HTML <BLINK><A HREF="[link]">Click here</A>

Маркъп съдържание има например във всеки doc файл, но то може да бъде обработено само от едно приложение (proprietary). Въпреки че такива файлове съдържат текст, за правилното визуализиране и работа с тях ни трябват точно определени програми. Това са binary файлове. Нуждата обаче от преносимост между различни платформи и приложения води до разработването на отворени маркъп езици.

През 1969 година е разработен един от първите маркъп езици – **Standard Generalize Markup Language (SGML)**, от който водят началото си най-разпространените маркъп езици днес – XML и HTML. Въпреки че от гледна точка на изчерпателност този език е много добър – с него може да се публикува почти всичко (математика, химия, UNIX документация, речници), той се оказва твърде сложен – средствата за обработка са много скъпи и сложни. Друг негов недостатък е, че не е разработен директно за WWW.

HTML (Hypertext Markup Language) се появява през 1989 и негов създател е Тим Бърнърс Ли. Неговият документен тип е описан чрез SGML. Използва се предефинирано множество от тагове (около 60). Има много частни разширения, което често води до несъвместимост на браузърите. Тъй като обаче таговете са предефинирани това води до ограничени възможности в експресивността на езика – повечето тагове са свързани със структурната организация на документа и неговото представяне, труден е за „разбиране" от компютри.

XML (eXtensible Markup Language) – позволява потребителят да си дефинира собствени тагове, затова е разширяем. Всеки документ съдържа четири типа информация:

- ✓ Съдържание – текст, графични елементи, звук
- ✓ Структура – на какви части се разделя съдържанието и как се подреждат те
- ✓ Външен вид – как се визуализират съдържанието и структурата на документа
- ✓ Меда данни – допълнителна информация за документа

XML йерархии, синтактични правила

Таг – това е маркъп, който се разделя от останалото съдържание с '<' и '>'. Съдържа името на елемент. Отварящ таг: <elementname>, затварящ таг </elementname>.

Елемент – означава се с отварящ и затварящ като между тях е съдържанието

- ✓ Именуване – винаги трябва да почва с буква или '_', след това може да има цифра, '-' или '.'. С ':' се разграничават името на пространството от имена и името на елемента. Имената не могат да започват с XML, XmL и т.н. Различават се малки и големи букви.

Атрибут – предоставят допълнителна информация за елемент

- ✓ Като им се задават стойност се огражда се с „” или ‘’
- ✓ Различават се главни и малки букви (case-sensitive)
- ✓ Character data or tokenized (value=”Blue Peter” character data, value=”blue” single token, value=”red green blue” tokens)

Елемент или атрибут

- ✓ Зависи от предназначението на конкретния документ.
- ✓ Трябва да се различи съдържанието от мета данните – съдържанието е това, което ще бъде видимо (представяно), останалата мета информация може да се зададе в атрибути.
- ✓ Правило: Като се махне всичкият маркъп, документът трябва да може да се чете и разбира.
- ✓ Използват се атрибути, когато се задава някакво свойство на даден елемент.
- ✓ Използват се атрибути за проста валидация на типа данни.
- ✓ Използват се елементи за сложна структурна валидация.

Структурата на XML документите е йерархична – всички елементи са вложени, т.е. не може елемент да има отварящ таг в рамките на един елемент и затварящ в рамките на друг. На всеки добре-структуриран XML документ може да се съпостави дърво. Има един елемент, който е корен на документа и всички останали са вътре в него. За всеки отварящ таг трябва задължително да има затварящ такъв.

План за създаване на XML документа:

- ✓ Въведение (prolog) - всеки XML документ започва с XML декларация – <?xml version="1.0" encoding="UTF-8" standalone=“yes"?>. XML декларацията има задължителен атрибут version за използваната версия на XML спецификацията и също така незадължителните encoding и standalone. Ако standalone е “yes”, то документът не зависи от други файлове. Кодирането на символите по подразбиране е UTF-16 или UTF-8.

Може да има и зададено стилово множество – <?xml:stylesheet type="text/css" href="s.css" **?>.** Също така може да се зададе и документният тип - <!DOCTYPE test SYSTEM "test.dtd">.

- ✓ Инстанция – това е йерархията от елементи
- ✓ Допълнения:
  - коментари – <!—comment -->
  - инструкции към процесора (<?PITarget Status="draft" ?>) – това е информация, необходима от външно приложение. XML декларацията също е инструкция към процесора.

Текстовото съдържание на елементите може да бъде два типа **PCDATA** (parsed character data) и **CDATA** (character data). В първия случай (PCDATA) това не се обозначава по никакъв начин и маркъп символи в съдържанието се разпознават от парсера. Ако в съдържанието на някой елемент искаме да включим знаците, използвани за маркъп, то трябва да ползваме специални последователности от символи:

- ✓ & - &
- ✓ < - <
- ✓ > - >
- ✓ ‘ - '

- ✓ “ - "

Ако искаме да избегнем това обаче, може да ползваме CDATA – започва с <![CDATA[и завършва с]]>. Съдържанието в такава секция не интерпретира по никакъв начин и се подава на приложението както е.

Пространства от имена

Поради използването на **едни и същи имена**, придавайки им **различно значение** (имат различна семантика), се налага употребата на пространства от имена за елементите и атрибутите. Пространство от имена е абстрактна нотация за група от имена. В рамките на един XML документ дадено пространство от имена може да се означава с префикс (ако не е това по подразбиране, когато префикс няма).

Qualified names – префикс:име

Уникална идентификация на пространството от имена – **URL**. Обосновка: може за различни пространства от имена в различни документи да се ползва един и същи префикс.

Задават се като **атрибут: xmlns**. Може да се зададе на всеки елемент. В един документ може да има информация, дефинирана в няколко пространства от имена.

XML валидация чрез Document Type Definitions (DTD)

Цели на валидирането

Когато се създават XML документи, те могат да се категоризират в групи от сходен тип в зависимост от елементите и атрибутите, които съдържат. Елементите и атрибутите, които се срещат в един документ, определят неговия речник. Използвайки DTD, лесно могат да се валидират XML документи спрямо дефиниран речник от елементи и атрибути. За да се провери дали съдържанието на даден XML документ е валидно според декларациите в съответния DTD документ, може да се ползва XML парсер.

DTD структура

DTD е файл, който съдържа формална дефиниция на конкретен тип документ.

Може да се намира вътре в XML документа, в отделен файл или и двете – при наличие и на двата вътрешният доминира.

Вътре в документа (само едно вътрешно DTD):

```
<!DOCTYPE MyDoc [
        <!-- DTD appears here -->
        <! … >
        <! … >
]>
<!-- Rest of XML file -->
<MyDoc> ……
</MyDoc>
```

Референция към **външен DTD файл** (за преизползване; декларациите, специфични за XML документа се оставят вътре в него):

```
<!DOCTYPE MyDoc
        SYSTEM "./MyDoc.dtd" [
        <!-- Extra declarations -->
```

```
        <! … >
        <! … >
    ]>
    <!-- Rest of XML file -->
```

SYSTEM – локални файлове и URL

PUBLIC – за публични каталози

DTD декларациите са инструкции към XML процесора. Означават се <! .. > или <! … [<! … >]>.

Документен тип – <!DOCTYPE My_XML_Doc […]>

- ✓ посочва името на елемента корен на XML документа
- ✓ след това в квадратните скоби се декларират останалите елементи

Character data – <![CDATA[…]]>

Семантични единици – <!ENTITY … >

- ✓ Семантична единица – един XML документ може да бъде разделен в няколко отделни файла, като всяка единица информация се нарича семантична единица
- ✓ всяка семантична единица има име, с което се идентифицира
- ✓ може да се реферира с entity reference
- ✓ видове семантични единици
    - Built-in – не е необходимо да се декларират (това са & < и т.н.)
    - Character – те също не се декларират; обикновено се ползват за символи, които трудно се изписват; реферират се или © , или с кода от Unicode представянето (&#x00A9)
    - General – декларират се в DTD; най-често се ползват за деклариране текст, който може лесно да бъде сменян; реферират се с &име;
    - Parameter – те са за употреба само в DTD; тук текстът може да съдържа части от DTD декларации за разлика от другите entities; след ключовата дума ENTITY се поставя % следван от името на entity и после е съдържанието; реферират се с %име.
- ✓ Външни entities – текстови: могат да се посочват с SYSTEM или PUBLIC идентификатор, binary: същото като текстовите + може с ключовата дума NDATA да се задава видът на файла

Нотация – <!NOTATION … >

- ✓ Задава описание на външна не-XML семантична единица
- ✓ Използва се в комбинация с NDATA

Елемент – <!ELEMENT … >

- ✓ използва се за декларация на нови елементи и модела на тяхното съдържание
- ✓ състои се от три части – декларацията ELEMENT, името на елемента и модела на съдържанието
- ✓ съдържанието може да бъде
    - елементно – (name, e-mail*), като всеки реферирян елемент също трябва да има дефиниция в DTD
    - смесено – (#PCDATA|a|b)*
    - празно – EMPTY
    - всякакво – ANY
- ✓ Индикатори за брой срещания (cardinality, quantity indicators):

- + - [1..*]
- * - [0..*]
- ? - [0..1]

✓ Последователност (sequence) – задава се ‘,’, и избор (choice) – задава се с ‘|’

Атрибути – <!ATTLIST … >

✓ Атрибутите могат да бъдат прикрепени към декларацията на елементите или да бъдат в отделна декларация.

✓ Декларацията се състои от три части:

- Ключовата дума ATTLIST
- Името на елемента, за който са тези атрибути
- Списък с декларациите на атрибутите – за всеки атрибут се задават име, тип и декларация на стойността.

  Пример: source CDATA #IMPLIED

  Декларацията на стойността може да бъде - стойност по подразбиране, фиксирана стойност, изисква ли се (Is required), is implied (or is optional).

  Заб: Декларациите на пространства от имена, които ще се ползват в XML документа, също трябва да се дефинират като атрибут, въпреки че според препоръката за пространствата от имена на W3C те са декларации, а не атрибути. Това се получава, тъй като тази препоръка излиза след завършването на DTD синтаксиса.

✓ Възможни типове на атрибутите:

- **CDATA –** показва, че атрибутът има стойност от символи (character data), парсерът не интерпретира маркъп символите
- **ID –** показва, че стойността на атрибута уникално идентифицира елемента
- **IDREF –** стойността на атрибута е референция по ID към уникално идентифициран елемент
- **IDREFS –** стойността на атрибута е списък от разделени с празно място IDREF стойности
- **ENTITY –** стойността на атрибута е референция към външна (unparsed) семантична единица. Това може да бъде картинка, MP3 или друго binary.
- **ENTITIES –** списък от ENTITY стойности разделени с празно място
- **NMTOKEN –** стойността е name token – това е стринг от група стандартни стрингови имена.
- **NMTOKENS –** списък от NMTOKEN стойности разделени с празно място
- **Enumerated List** – може да се декларира енумерация от възможни стойности. *Пример*: (alpha|num).

<![INCLUDE[…]]> и <![IGNORE[…]]>

✓ Могат да се ползват като switch включване и изключване на декларации

Недостатъци на валидацията с DTD

✓ Синтаксисът не е XML

✓ XML пространства от имена – всеки път, когато елемент или атрибут се добавят в DTD трябва да се включи и пространството от имена към името. Освен това в DTD пространствата от имена се третират като атрибути, въпреки че според препоръката на

W3C те са декларации. Това може да доведе до някои объркания (например ако се съединят два XML документа с едно и също пространство от имена, което обаче има различен префикс в различните документи).

- ✓ Ограничени са типовете данни – единствените типове в DTD се използват при декларацията на атрибути и те са крайно недостатъчни. Няма и начин да се ограничи типът на текстов елемент – например не може да се зададе даден елемент да бъде число.
- ✓ Ограничения в модела на съдържанието – не могат да се задават връзки от тип наследяване; не могат да се задават ограничения в броя срещания на даден елемент (например от 1 до 10).

XML валидация чрез XML Schema

Схема е всеки документ, който се използва за моделирането на структурата на нещо. Днес терминът XML Schema се използва по-конкретно за W3C XML Schema технологията. XML Schema, подобно на DTD, позволява да се опише структурата на един XML документ. Самата XML Schema се валидира от DTD.

Спецификации

Спецификацията на XML Schema е разделена на три части.

- ✓ Част 0: Въведение – включва въведение в основните концепции, не е със задължителен характер.
- ✓ Част 1: Структури – дефинира структурите, които се ползват в XML Schema
- ✓ Част 2: Типове данни – специфицира какви типове могат да се ползват за елементи и атрибути

Структури

- ✓ Дефиниции на типове - <simpleType>, <complexType>
- ✓ Декларации на атрибути - <attribute> (декларациите на атрибути винаги са последни, след декларациите на елементи)
- ✓ Декларации на елементи - <element>
- ✓ Дефиниции на групи от атрибути - <attributeGroup>
- ✓ Дефиниции на групи - <group>
- ✓ Декларации на нотации - <notation>
- ✓ Анотации - <annotation>

Заб: Декларациите се правят на нещата, които ще използват в инстанция на XML документа, а дефинициите на нещата, които се ползват в самата схема (например дефинира се нов тип).

Глобални и локални декларации – глобалните са тези, които са директни деца на <schema> елемента, а локалните нямат като родител този елемент. Глобално декларирани елементи могат да се ползват като се зададе стойност на ref атрибута на декларацията element (ако targetNamespace не е namespace по подразбиране, то елементът трябва да се реферира с qualified името си). Въобще всички глобални декларации отиват в target пространството от имена и когато се ползват трябва да се дава неговият префикс (ако то не е пространството от имена по подразбиране).

<schema>

- ✓ Коренът на схема документа

Атрибути:

- ✓ targetNamespace – това е пространството от имена на елементите, които ще бъдат в инстанциите

<element>

Атрибути:

- ✓ minOccurs, maxOccurs – брой срещания (не са позволени в глобално декларирани елементи, могат да се ползват при реферирането на глобално дефиниран елемент)
- ✓ default – позволява задаването на стойност по подразбиране за елемента; ако елементът няма стойност, то той ще се третира от валидатора със стойността по подразбиране
- ✓ fixed **–** за задаване на фиксирана стойност

<any> - позволява даден XML документът да включва елементи, които не са зададени в схемата му; използва се само в модела на съдържанието, не може да е глобална декларация

За задаването на модела на съдържанието на елемент може да се ползват

<complexType> - с него се задава моделът на съдържанието на елемент; първо се задават винаги декларациите на елементи, а след това на атрибути

Атрибути:

- ✓ mixed – true или false; дали може да има само текст или текст и елементи
- ✓ name – *заб:* локалните декларации не могат да имат име (такива сложни типове се наричат анонимни); глобалните декларации за разлика, винаги трябва да имат име

<group> - позволява да се групират елементи и така получената група да се преизползва на различни места; използва се само за елементи, не са позволени декларации на атрибути; когато трябва да се ползва дадена глобално дефинирана група, се използва референция към нея; редът на елементите в групата се задава с помощта на декларациите <sequence>, <choice>, <all>

<sequence> - съдържа декларации на елементи, които трябва да присъстват в XML документа в посочения ред

<choice> - съдържа декларации на елементи, от които точно един може да присъства в XML документа

<all> - съдържа декларации на елементи, които трябва да присъстват в XML документа независимо в какъв ред

Горните четири декларации могат да се ползват по различни начини, за **да се зададе модела на съдържанието** – той може да бъде зададен вътре в декларацията на самия елемент (inner content models), element declarations, element wildcards.

В сравнение с DTD схемата дава много повече възможности за **задаване на уникалност**:

- ✓ може да се зададе съдържанието на даден елемент да е уникално
- ✓ може да се зададе атрибути, които не са ID, да са уникални
- ✓ може да се зададе комбинация от елементно съдържание и атрибути да бъде уникална
- ✓ прави се разлика между уникален и ключ
- ✓ може да се декларира област от документа, в която нещо е уникално

<key> - заедно с <keyref> позволява да се дефинира връзка между два елемента; вътре се дефинират <selector> - посочва за кой елемент е ключът и <field> - посочва кой връх е ключът

<unique> - отново се задават <selector> и <field>; стойността на нещо, което е unique обаче може да е и null

<attribute> - използва се за декларация на атрибути; обикновено се среща или в <attributeGroup>, или в <complexType>; може също да се ползва и в <extension> или <restriction> при дефиниране на производен тип.
<anyAttribute> - позволява XML документът да се разширява с атрибути, които не са специфицирани в схемата му
<attributeGroup> - позволява декларацията на група от атрибути; ако е глобална трябва да има име
Има две основни декларации за работа с повече от един XML Schema документи:
<import> - позволява да се внесат глобални декларации от други XML Schema документи като се зададат атрибутите namespace – target namespace на внасяната схема, и schemaLocation – местоположението на внасяната схема; винаги е глобална декларация – директно дете на <shcema>
<include> - много е подобна на <import> декларацията, но разликата е, че внесените глобални декларации от новата схема директно се причисляват към target namespace-а; затова има само атрибут schemaLocation и с него се посочва къде е схемата, от която да се внесат глобалните декларации

Типове данни

Вградени (built-in) – те включват 2 групи:

- ✓ примитивни (основни) – не са дефинирани, използвайки други типове; служат като основа за дефинирането на всички останали типове
  - string
  - boolean
  - float
  - double
  - decimal
  - timeDuration
  - recurringDuration
  - binary
  - uriReference
  - ID
  - IDREF
  - ENTITY
  - NOTATION
  - QName
- ✓ вградени производни (built-in derived) – използвани са други примитивни и производни типове за да бъдат дефинирани (но са вградени в схемата) /например positiveInteger, recurringDay и т.н./

Дефинирани от потребителя (user-defined)
<simpleType> - използва се за дефиниране на потребителски тип данни, като новият тип винаги се базира на вече съществуващ тип данни – или от вградените типове, или вече дефиниран от потребител друг тип. За целта се използват: <restriction>, <list>, <union>
<restriction> - типът, който се получава, е подмножество на основния тип (<restriction base=”name of the simpleType you are deriving from”>); използва се с фасетите за ограничения

<list> - <list itemType=”name of simpleType used for validating items in the list”>; може и вътре да се дефинира simpleType (типът на нещата от списъка)
<union> - позволява стойностите в документа инстанция да се валидират спрямо няколко типа едновременно (<union memberTypes=”whitespace separated list of types”>)

Фасети

Фасет е свойство или характеристика на <simpleType>. (A *facet* is a single property or trait of a <simpleType>). Фасетите на даден тип могат да се ползват, за да се ограничи неговото множество от стойности. С помощта на фасетите потребителят може да си дефинира нови типове от вградените такива.
Фундаментални – не могат да бъдат променяни; дефинират типа (няма ги в Beginning XML 4th)

- ✓ Equal
- ✓ Order
- ✓ Bounds
- ✓ Cardinality
- ✓ Numeric

Фасети за ограничения – позволяват да се ограничи множеството от стойности; ползват се в <restriction> (в лекциите са различни от тези в Beginning XML 4th, тук съм сложила от книгата, защото мисля, че са по-актуални)

- ✓ minExclusive – Allows you to specify the minimum value for your type that excludes the value you specify
- ✓ minInclusive – Allows you to specify the minimum value for your type that includes the value you specify
- ✓ maxExclusive – Allows you to specify the maximum value for your type that excludes the value you specify
- ✓ maxInclusive – Allows you to specify the maximum value for your type that includes the value you specify
- ✓ totalDigits – Allows you to specify the total number of digits in a numeric type
- ✓ fractionDigits – Allows you to specify the number of fractional digits in a numeric type (e.g., the number of digits to the right of the decimal point)
- ✓ length – Allows you to specify the number of items in a list type or the number of characters in a string type
- ✓ minLength – Allows you to specify the minimum number of items in a list type or the minimum number of characters in a string type
- ✓ maxLength – Allows you to specify the maximum number of items in a list type or the maximum number of characters in a string type
- ✓ enumeration – Allows you to specify an allowable value in an enumerated list
- ✓ whitespace – Allows you to specify how whitespace should be treated within the type
- ✓ pattern – Allows you to restrict string types using regular expressions

Сравнение с DTD

- ✓ Ползва се основният XML синтаксис, докато DTD дефинира свой отделен
- ✓ Поддържа изцяло препоръката за пространствата от имена
- ✓ По-експресивен синтаксис от DTD
- ✓ Повече поддържащи средства (DOM, браузъри)

- ✓ DTD поддържа само 10 типа данни, докато схемата 37 вградени и дава възможност също за дефиниране на нови
- ✓ По-добър модел на съдържанието – например може да се задава по-точно ограничението за брой срещания на даден елемент; могат много неща да се преизползват с помощта на дефинирането на глобални типове, групи от елементи, групи от атрибути, елементи
- ✓ Не само глобални, но и локални имена
- ✓ Неограничена възможност за разширяване
- ✓ Повече речници (базирани на използването XML пространства от имена)
- ✓ В XML Schema не може да се дефинира ENTITY
- ✓ DTD може да се вгради директно в документа, който се валидира, докато XML Schema е винаги в отделен файл

Използване на XSLT (eXtensible StyleSheet Language Transformations) и XPath за алокиране, манипулиране и представяне на XML съдържание.

XML документите обикновено са предназначени предимно за компютърна обработка, а не да бъдат четени от хора. Често се налага обаче XML съдържанието да се представи в различен вид в зависимост от това за кого или какво е предназначено. Затова W3C разработват спецификацията XSL – Extensible Stylesheet Language. По време на разработката се оказва, че се опитват да покрият две различни функции в една спецификация – да дефинират елементи за представяне на съдържание и да дефинират синтаксис за трансформиране на елементите и документната структура. Затова спецификацията се разделя на две – XSL Formatting objects (или само XSL) и XSL Transformations. Първото е език за форматиране на XML данни за представянето им на екран, хартия или друг медиен носител. Второто е език за трансформиране на XML документи в други документи.

XSLT

XSL процесорът може да **трансформира** един входен XML документ в XML, HTML или текстов изходен документ, като използва правилата, зададени в друг XML документ (XSLT файл). За разлика от JavaScript XSL е декларативен, а не императивен език. На входния документ се съпоставя входно дърво, а на изходния – изходно. Може да се добавя текст преди и след дадено съдържание, да се махат, създават и пренареждат елементи.

В сравнение с DOM възможностите са по-ограничени. Друга основна разлика е, че XSL е скриптов език, а DOM програмен интерфейс.

Client-side XSLT – ако браузърът поддържа XSLT, то документът може да се превърне в XHTML директно през него. Също така може да се ползва JavaScript за извикване на трансформацията – позволява да се прави проверка за браузъра и да се ползват различни стилови множества спрямо различните браузъри или изискванията на потребителя

Server-side XSLT - Ако обработката на входния XML файл е тежка или ако браузърът не поддържа XSLT, тя може да се извършва на сървърната страна.

Има дефинирани **34 елемента**, но дори само с три от тях могат да се правят полезни стилови множества (style sheets) – stylesheet, template, apply-templates

<xsl:stylesheet> - коренът на документа

<xsl:template> – специфицира правило за трансформация; match атрибутът приема XPath израз; атрибут mode. Темплейтът ще бъде приложен, ако някой връх във входното дърво отговаря на match атрибута.
<xsl:import> - използва се, за да се включат няколко XSLT файла
<xsl:value-of> – използва се, за да се преобразува обектът, посочен в select атрибута, в string (не е рекурсивна функция); ако е посочен елемент, то се взима неговото съдържание и всички елементи, които съдържа
<xsl:apply-templates> – използва се в даден темплейт за извикване на други темплейти; предизвиква рекурсивна обработка на всички потомци на елемент, който е бил match-нат
<xsl:output> - дава контрол върху изходния документ; слага се директно след <xsl:stylesheet>
<xsl:sort>
<xsl:number> - за автоматично номериране
Атрибути:

- ✓ level – ‘single’, ‘any’, ‘multiple’
- ✓ count – “list-1|list-2”
- ✓ format – “1.A”
- ✓ from – “3”
- ✓ grouping-separator – “,”
- ✓ grouping-size – “3”
- ✓ value – “position()”

<xsl:variable> - могат да се декларират променливи; за да се достъпи стойност на променлива се ползва ‘$’ (<xsl:value-of select=”$colour”/>); също така стойността на променливата може да се ползва и в елементи от резултатното дърво (<ajr:glyph colour=”{$colour}”/>). По този начин могат да се добавят прости константи (например да си дефинираме числото пи).
<call-template> - позволява прилагането на даден темплейт като се задава неговото име (<xsl:call-template name=”CreateHeader”/>)
<xsl:param> - специални променливи; позволява да се задават параметри на даден темплейт (стойността, посочена при декларацията на този елемент се счита по подразбиране). При извикването на темплейта с <call-template> може да се ползва елемента <xsl:with-param> и там да се зададе нова стойност. Стойностите на параметрите могат да се достъпват по същия начин като стойностите на променливите.
<xsl:element> - използва се за създаване на нови елементи и позволява динамично създаване на елементи (например съдържанието на елементите от входното дърво може да служи за имена на елементите от изходното дърво или атрибутите да се превърнат в елементи).
<xsl:attribute> - за създаване на атрибути
<xsl:attribute-set> - за създаване на групи от атрибути
<xsl:copy> - копира съдържанието на контекстния връх без децата и атрибутите; има атрибут use-attribute-sets, в който могат да се посочат групи от атрибути за копирания елемент.
<xsl:copy-of> - може да копира фрагменти от входното дърво без да се загубват атрибутите и вложените елементи (дълбоко копие)
<xsl:for-each> - позволява да се итерира по елементите на дадено множество от върхове
<xsl:if> - съдържа темплейт, който ще се приложи само ако условието в атрибута test е изпълнено
<xsl:choose> - използва се в комбинация с <xsl:when> и <xsl:otherwise> за изразяване на условни твърдения (multiple conditional statements)

XPath

XPath – схема за локализиране и идентифициране на подструктури в документи; използва се от други XML технологии като XSLT, XPointer, XQL като средство за локализиране на определени части от XML съдържанието.

XPath модел на данните – при XPath повечето части на един сериализиран XML документ се представят като върхове на дърво. Коренът на дървото е самият XML документ – отбелязва се с '/'. За всеки елемент, атрибут, инструкция към процесора или коментар има съответен връх в дървото. Също така има текстови върхове, представящи текстовото съдържание на елементите. XPath се възползва от последователния и йерархичен характер на XML документите, за да открие елементи според техния контекст (например къде в йерархията се намира даден елемент)

Контекстен възел – селектира се с .; това е текущата секция в XML документа, от където започва XPath изразът

XPath връх – може да бъде всяка част от XML документ – елемент, атрибут, инструкция към процесора или нещо друго

Корен на документа (document root) – не е първият елемент, а самият XML документ; означава се с '/'

XPath израз – текстов стринг, с който се избира даден елемент, атрибут, инструкция или текст; може да се намира в URL или като стойност на атрибут (http://abc.com/getQuery?/book/intro/title или <xsl:pattern match="chapter/title">… </xsl:pattern>). Изразите локализират нещата по тяхното място в XML йерархията

4 типа изрази – булеви, множества от върхове, числа, стрингове (това са типовете, които могат да бъдат върнати вследствие на оценката на XPath израза)

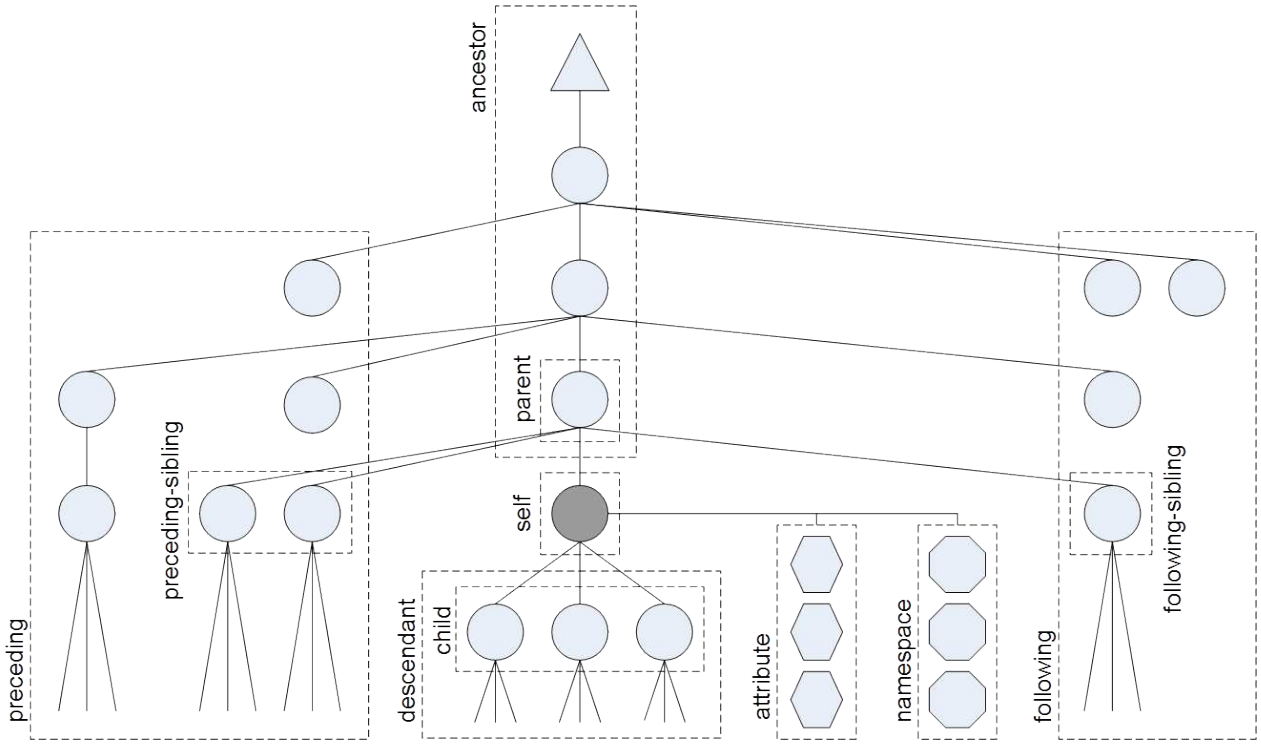
Location path – повечето XPath изрази връщат множество от върхове, тези изрази се наричат location paths. Пътищата, посочващи местонахождението (location paths), са съставени от стъпки. Всяка стъпка е съставена от ос, връх и предикат (опционален). Когато оста липсва, подразбира се child.

- ✓ Relative paths **–** започват от контекстния връх
- ✓ Absolute paths **–** започват от корена

Видове върхове

- ✓ Корен
- ✓ Връх на елемент
- ✓ Връх на атрибут – върхът на елемента, към който е този атрибут, се нарича връх родител. Всеки връх на атрибут има име и стойност.
- ✓ Текстов връх
- ✓ Връх за пространство от имена
- ✓ Връх за коментар
- ✓ Връх за инструкция към процесора

13 XPath оси – използват се така: име_на_оста::име_на_връх. Оста на контекстния връх се нарича self. Той може да се достъпи по два начина: '.' и self::node.



Заб: На картинката липсват descendant-or-self и ancestor-or-self. С тях не се селектират братя и сестри.

Рекурсивен оператор за достъп до всички потомци – '//'. Пример: chapter//para ще селектира всички para елементи, които са по-ниско в йерархията от chapter. Несъкратеният запис е: child::chapter/descendant-or-self::node()/child::para. Пример: За избиране на всички para елементи в текущия – .//para или descendant-or-self::node()/child::para.

С '..' се селектира родителят на контекстния връх, може да се достъпи и с parent оста.

С '@' се селектира стойността на атрибут

Функции

- ✓ node()
- ✓ text()
- ✓ position()
- ✓ last()
- ✓ count()
- ✓ id()
- ✓ concat()

Предикати – задават се с начупени скоби ([]). Предикат може да се сложи след всяка стъпка в XPath израза, като на една стъпка могат да бъдат зададени и няколко предиката на стъпка.

Използване на DOM (Document Object Model) и SAX (Simple API for XML) за обработка на XML документи – основни интерфейси на DOM и SAX и начини за използването им. Сравнение между DOM и SAX.

Има три основни стъпки при използването на XML парсер:

1. Създаване на обект от типа на парсера

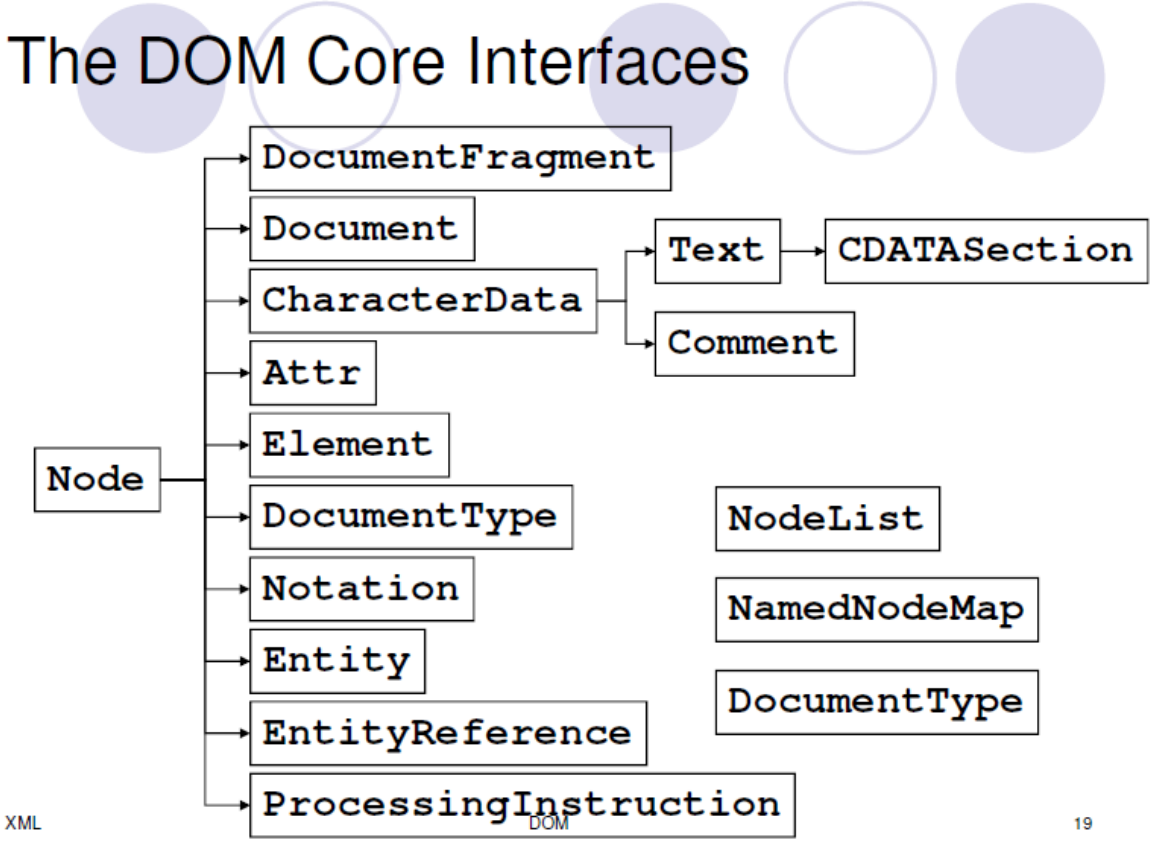
2. Подаване на XML документа към парсера
3. Обработка на резултатите

В общия случай генерирането на XML съдържание е извън възможностите на парсерите, но има и такива, които могат да поддържат и такава функционалност.
Двата главни програмни интерфейса (APIs) за парсване на XML са SAX и DOM.

DOM (Document Object Model)

DOM (Document Object Model) е W3C стандарт, който дефинира стандартен начин за работа с документи като XML и HTML. XML DOM дефинира обекти и свойства за всички XML елементи, както и методи (интерфейс) за достъп до тях. Платформено-независим модел. Използва се от програмистите основно като начин за манипулация на съдържанието на XML документи – може да се създава такова съдържание, да се навигира, да се модифицират, добавят и изтриват части от XML документи. Характерно за DOM е, че в паметта се зарежда целият документ, като за целта се парсва и му се съпоставят съответните обекти с техните свойства.
В DOM документът се представя като йерархия (дърво) от node обекти.
Спецификацията включва основна задължителна част от интерфейси, която трябва да бъде налична във всички имплементации, и различни разширения – за работа конкретно с XML, HTML.
Има три нива – всяко надгражда следващото; в DOM Level 2 е добавена функцията "getElementById", също така поддръжка на XML namespaces and CSS. DOM Level 3 е текущото ниво на спецификацията, добавена е поддръжка на XPath, обработка на събития от клавиатурата и интерфейс за сериализиране на XML документи.

Основни DOM интерфейси



Кой какви деца може да има:

- ✓ Document – Element (maximum of one), ProcessingInstruction, Comment, DocumentType (maximum of one)
- ✓ DocumentFragment – Element, ProcessingInstruction, Comment, Text, CDATASection, EntityReference
- ✓ DocumentType – no children
- ✓ EntityReference – Element, ProcessingInstruction, Comment, Text, CDATASection, EntityReference
- ✓ Element – Element, Text, Comment, ProcessingInstruction, CDATASection, EntityReference
- ✓ Attr – Text, EntityReference
- ✓ ProcessingInstruction – no children
- ✓ Comment – no children
- ✓ Text – no children
- ✓ CDATASection – no children
- ✓ Entity – Element, ProcessingInstruction, Comment, Text, CDATASection, EntityReference
- ✓ Notation – no children

Node – това е основният тип данни за целия документен обектен модел. Представя един връх в документното дърво. Методите му се разделят в три категории:

- ✓ Характеристики на самия връх – име, тип, стойност
- ✓ Разположение в дървото и достъп до съседни върхове – родители, братя и сестри, деца, предшественици, потомци
- ✓ Модификация на върхове – редактиране, изтриване, пренареждане на върхове деца

Document – представя целия XML документ (корена на дървото); съдържа методи фабрики за създаване на останалите обекти (елементи, текстови върхове, коментари, инструкции за процесора.

DocumentFragment – фрагмент от документа може временно да се съхрани в такъв връх. Обектите от тип са „леки” (lightweight) и могат да се ползват примерно при разместване на части от XML документа. Въпреки че за целта би могло също да се ползват и обекти от тип Document, то тези обекти могат да бъдат „тежки” в зависимост от имплементацията. Някои методи (например за добавяне на деца на някой връх) могат да приемат DocumentFragmen обекти като аргументи. По този начин децата на обекта DocumentFragment се добавят към списък от деца на върха. DocumentNode обектите не са задължително добре-структурирани XML документи – например може да има само едно дете и то да е връх от тип Text.

Element – два типа методи общи и свързани с атрибутите на даден елемент. Методите, предоставящи интерфейс към атрибутите на елемента са относително просто – трябва да знаеш името на атрибута, не могат да се различат стойността по подразбиране, зададена в DTD, и реалната стойност в XML документа, не може да се определи кога типът на атрибута е [String] (Need name of attribute; Cannot distinguish between default value specified in DTD and given in XML file; Cannot determine attribute type [String]). По-добре е да се ползва методът, който връща NamedNodeMap от обекти Attr с атрибутите на дадения елемент.

Attr – предоставя интерфейс към обекти, съдържащи информация за атрибути

- ✓ String getName();
- ✓ String getValue();
- ✓ Void setValue(String value);

- ✓ Boolean getSpecified(); //показва къде е зададена стойността – дали в самия XML документ или е стойността по подразбиране от DTD

Референциите към Entity са деца на атрибутите. Атрибутите не се считат за част от дървото и затова parentNode, previousSibling и nextSibling имат стойност null за обектите от тип Attr. Обекти от този тип могат да се създават с метод фабрика в интерфейса на Document: Attr createAttribute(string Name);

CharacterData – има полезни методи за обработка на текст; не се използва директно, а през класовете наследници Text или Comment

- ✓ String getData() throws DOMException;
- ✓ void setData(String data) throws DOMException;
- ✓ int getLength()
- ✓ void appendData(String data) throws DOMException;
- ✓ String substringData(int offset, int length) throws DOMException;
- ✓ void insertData(int offset, String data) throws DOMException;
- ✓ void deleteData(int offset, int length) throws DOMException;
- ✓ void replaceData(int offset, int length, String data) throws DOMException;

Text – представя текстовото съдържание на обект от тип Element или Attr. Обикновено обектите от тип Text са деца на обекти от тип Element или Attr и винаги са листа в дървото. Има само още един метод в допълнение на интерфейса на CharacterData: Text splitText(int offset) throws DOMException;

За създаването на обекти от тип Text се използва метод фабрика от интерфейса на Document.

С извикването на normalize() метода на даден обект от тип Element се сливат всички текстови обекти, деца на елемента.

CDATASectioin – представя CDATA – текст, в който няма се интерпретира markup, разпознава се единствено “]]>”, който е краят на CDATA секцията. Няма добавени методи към интерфейса на CharacterData.

Comment – използва се да се представят коментарите в XML документа. Няма добавени методи към интерфейса на CharacterData.

ProcessingInstruction – представя декларациите на инструкции към процесора в XML документа. Тук името на върха е името на приложението, което трябва да се стартира, а стойността на върха е командата за приложението.

- ✓ String getDaga();
- ✓ Void setData(String data);
- ✓ String getTarget();

Entity – представя (represents) семантична единица (такава, която може да се парсва или не) в XML документа. XML процесорът може или да замени референция към семантична единица със самата нея, или да създаде връх от тип EntityReference. При семантични единици, които не могат да се парсват, референциите се запазват, а не се заменят със самата семантична единица. Към методите от интерфейса на Node са добавени още:

- ✓ String getPublicId();
- ✓ String getSystemId();
- ✓ String getNotationName();

Последният се ползва за семантични единици, които не могат да се парсват, в противен случай връща null. Entity, което може да се парсне, може да има деца, които представляват

стойността, с която може да бъде заменено entity. Всички семантични единици в даден XML документ могат да се вземат с getEntities() от интерфейса на DocumentType

EntityReference

Notation – всяка декларация на нотация в DTD се представя с Notation връх. Към методите на Node са добавени:

- ✓ String getPublicId(); //връща съдържанието на PUBLIC идентификатора
- ✓ String getSystemId(); //връща съдържанието на SYSTEM идентификатора

Всички нотации в даден документ могат да се вземат с getNotations() от DocumentType

NodeList –колекция от подредени обекти от тип Node

- ✓ int getLength();
- ✓ Node item(int index);

NamedNodeMap

- ✓ NamedNodeMap myAttributes = myElement.getAttributes();
- ✓ NamedNodeMap myEntities = myDocument.getEntities();
- ✓ NamedNodeMap myNotations = myDocument.getNotations();

Методи:

- ✓ int getLength();
- ✓ Node item(int index);
- ✓ Node getNamedItem(String name);
- ✓ Node setNamedItem(Node node) throws DOMException;
- ✓ Node removeNamedItem(String name) throws DOMException;

DOMImplementation – използва се, за да се определи нивото на поддръжка от DOM парсера

- ✓ hasFeature(String feature, String version);

DocumentType – съдържа DTD информацията за документа. В DOM 1.0 не се позволява промяна на този връх

DOM обектите могат да се сериализират, ако ще се предават отдалечено, за да се избегне повторно парсване на XML документа за създаване на обекти. Недостатъкът е, че обектите могат да бъдат по-големи от самия XML документ.

SAX (Simple API for XML)

- ✓ Разработен, за да позволи по-ефективна обработка и анализ на големи XML документи. Целта е да се разреши един от главните проблеми на DOM – за да се обхожда XML дървото, целият документ трябва да се зареди в паметта. Това води до заемане на голямо количество памет и също така отнема време за създаването на обектното дърво.
- ✓ Не се притежава от никой, възниква от една мейлинг листа (XML-DEV), която в момента се хоства от OASIS.
- ✓ Event-driven API
  - XML файлът се изпраща на SAX парсера
  - XML файлът се чете последователно
  - Парсерът нотифицира даден клас, когато се случи някакво събитие (включително грешки)
  - За обработка на събитията се имплементират методи от API-то
- ✓ SAX парсерът генерира събития:

- В началото и в края на документа
- В началото и в края на всеки елемент
- Когато намери символи в елемент
- Когато се натъкне на грешки
- Когато се натъкне на negligible whitespace
- В още няколко други случая

✓ Използва се callback механизъм за да се нотифицира приложението
✓ Всички SAX интерфейси се предполага, че са синхронни

- parse методите не трябва да връщат стойност преди парсването да е завършило
- readers трябва да чакат обработчика на събитието (това е callback) да върне резултат преди да продължат с докладването за следващото събитие

✓ Ползва се само за четене, но не и за създаване на XML съдържание
✓ Не е подходящ, ако трябва да се манипулира структурата на документа

Основни SAX интерфейси

DocumentHandler – за контролиране на събитията, свързани с обработката на документа; има го само в SAX1, в SAX2 отпада и е заменен от ContentHandler, който има поддръжка на пространства от имена.

ContentHandler – Използва се за получаване на основни събития от парсера. Има имплементация DefaultHandler (в SAX1 HadnlerBase) с празни имплементации на handler-ите; тя е за удобство, за да не се налага да се предоставят имплементации на всички методи от интерфейса, а само на тези, които са необходими в конкретното приложение. В DefaultHandler има имплементации на всички callbacks от четирите основни handler интерфейса:

✓ EntityResolver
✓ DTDHandler
✓ ContentHandler
✓ ErrorHandler

XMLFilterImpl – имплементация на ContentHandler; стои между XMLReader и обработчиците на събития в приложението – подава заявки към reader-а и събития към handler-ите без да ги модифицира, но подкласове могат да override-ват методите.

XMLReaderAdapter – wrapper за XMLReader – прави го като Parser от SAX1

XMLReader – интерфейс за четене на XML документ, използвайки callback функции. Въпреки името си, интерфейсът не разширява стандартния Java Reader интерфейс, защото тук става въпрос за различен тип действие от четенето на символни данни.

Позволява на приложението да задава и проверява характеристики и свойства на парсера, да регистрира обработчици на събития при обработката на документа и да инициира парсването на документа.

XMLReader замества отпадналия интерфейс Parser в SAX1. XMLReader има две съществени подобрения спрямо стария Parser интерфейс:

✓ Добавя стандартен начин за проверяване и задаване на характеристики и свойства на парсера.
✓ Добавена е поддръжка на пространства от имена.

void setContentHandler(ContentHandler handler)
void setDTDHandler(DTDHandler handler)

void setErrorHandler(ErrorHandler handler)
void setEntityResolver(EntityResolver resolver)
Парсването се започва с метода parse().
Ако приложението не регистрира DTD handler, то всички DTD събития просто се игнорират.
Handler-ите могат да се сменят по време на самото парсване и SAX парсерът веднага започва да ползва новите.
Когато има нещо, за което се генерира събитие, четенето спира и се изчаква обработчикът да върне резултат, след което се продължава отново.

lputSource

- ✓ void setByteStream(InputStream byteStream)
- ✓ void setCharacterStream(Reader characterStream)
- ✓ void setEncoding(String encoding)

EntityResolver – приложението не е наясно с физическата структура на XML данните. Парсерът има entity manager, който скрива сложността на физическата структура от приложението, което вижда всичко като един поток от данни.
Can intercept references to entities by implementing the EntityResolver interface
Ако приложението има нужда да имплементира собствена обработка на външни entities, трябва да имплементира този интерфейс и да регистрира инстанция, използвайки setEntiryResolver. Когато парсерът се натъкне на entity, той подава системния и/или публичния идентификатор на приложението: public InputSource resolveEntity(String publicID, String systemID)

ParserFactory – помощен клас, предоставя удобни методи за динамично зареждане на SAX парсери

- ✓ makeParser()
- ✓ makeParser(String className)

21. Използване на XML за структуриране, валидация, обработка и представяне на документно съдържание.

Добре структуриран XML

Основни концепции:

- XML таг – markup на документа, отделен от останалата част със символите < и >. Съдържа името на елемента
  - Start tag <elementname>
  - End tag </elementname>
- XML елемент – информацията между отварящ и затварящ таг включително. Информацията между таговете се нарича съдържание на елемента.
- XML Markup – състои се от таговете в документа.
- XML документ - съдържа текст под формата на един или повече елементи.
- XML атрибути – в допълнение към тагове и елементи, XML документите могат да съдържат и атрибути. Атрибутите са двойки име/стойност (стойността е задължителна дори да е празен стринг), асоциирани с елемент. Те се добавят към отварящия таг - <name nickname=”Smth”> и стойностите им задължително се поставят между ‘’ или „“.
- Processing instructions - <?....?> - представят информация, необходима на външни приложения.
- XML декларация - <?xml version=’1.0’ encoding=’UTF-16’ standalone=’yes’ ?> - указва версията на XML спецификацията, към която се придържа документа, енкодинга му и дали с него е асоцииран външен DTD.

XML Йерархии:

XML документите представляват йерархични дървета. Всеки документ може да бъде представен като дърво, в което отделните възели са XML елементи.
Root елемент – всеки XML документ има точно един елемент, който няма родител. Той е първият елемент в документа и съдържа всички други елементи.
Всеки XML документ съдържа следното:

- Prolog – stylesheet & document type декларации
- Instance – йерархията от елементи
- Additions – коментари <!—comment--> и processing instructions <?PITarget Status=”draft”?>

Синтакични правила

XML документите трябва да се придържат към определени правила, за да бъдат добре структурирани:

- Всеки отварящ таг трябва да има съвпадащ затварящ таг или да бъде self-closing таг.
- Таговете не могат да се застъпват, елементите трябва да са properly nested.
- XML документите могат да имат само един root елемент.
- Имената на елементите трябва да спазват XML конвециите.
- XML е case sensitive.
- XML запазва whitespace в PCDATA.
- В съдържанието на документите не се позволяват запазените от XML символи. Те се заменят с predefined entities или се поставят в <![CDATA[.....]]> секция.

XML пространства от имена

Namespace/пространство от имена – абстрактна нотация (категория) за група от имена. Едно име може да принадлежи само към една група. Създадени са с цел да се осигури уникалност между XML

елементите. За име на namespace в XML обикновено се използва URI. Във всеки документ може да се съдържа информация за множество пространства от имена.
Декларация на namespace – за да се асоциира определен елемент с namespace в деклаацията му се добавя атрибут xmlns - <pers:person xmlns:pers="http://www.wiley.com/pers"/>. Тази декларация важи за елемента, в който е поставена, както и за всички негови наследници. В един елемент могат да бъдат деклрирани няколко namespace-а.

XML валидация чрез Document Type Definitions (DTD)

Цели на валидирането

Синтактичните правила в XML се налагат от парсерите, но често е необходимо да има начини за разграничаване на различните XML-базирани езици(речници). Всеки документ обикновено се опитва да следва определен език/речник и са необходими методи за определяне на нивото на съответствие. За тези цели се използва валидация чрез DTD или XML Schema.
XML документ е валиден, ако съдържанието му отговаря на дефиницията на позволените елементи, атрибути и други части от документа.
Целта на един DTD е да декларира всички имената на всички елементи и атрибути, които се използват в XML документите, отговарящи на това DTD, типовете им, видовете елементи, как те се изолзват заедно и йерархията на документа. DTD-тата представляват шаблони за markup-а на документите и съдържат формалната дефиниция на съответния тип документ. DTD може да бъде механизъм за стандартизация и манипулация на документи/данни и обмяната им.

DTD структура

DTD се съхраняват във външни файлове, или в самия XML файл или и двете.
DTD се състоят от:

- Document Type Declaration (DOCTYPE) – информира парсера, че с документа има асоцииран DTD. Задължително трябва да е поставена в началото на документа
- Тяло на DTD – поставя се след DOCTYPE декларацията и съдържа декларациите на елементи, атрибути, entities и нотации.
- Затваряне на декларацията на DTD с "]>"

```
<!DOCTYPE name [                                    // Document Type Declaration
        <!ELEMENT name (first,  last)>              // Body
        <!ELEMENT first (#PCDATA)>
        <!ELEMENT last (#PCDATA)>
]>                                                  // End of DTD
```

DTD синтаксис

DTD-тата се състоят от декларации на елементи, атрибути към тях, entity-та и нотации. Форматът на декларациите е <! ...> или <! … [<! … >]>

Document Type Definition

<![DOCTYPE root_element SYSTEM file […] > и *<![DOCTYPE root_element PUBLIC fpi (system_id)[…] >* указва на парсера, че с XML документа е асоцииран DTD. Съдържа <![DOCTYPE, името на root елемента на документа, и идентификатор към файла с DTD-то. SYSTEM и име на файл указва, че DTDто, асоциирано с документа, се намира във file, а PUBLIC – указва публично DTD, достъпно на fpi и опционално задава локално DTD, което да се използва, в случай че публичното не е достъпно.

Декларация на елемент

<!ELEMENT first (#PCDATA)>

Състои се от три основни части

- ELEMENT декларация
- Име на елемент
- Модел на съдържание на елемента – дефинира допустимото съдържание на елемента. Има 4 вида модели на съдържание:
    - Елемент *<!ELEMENT name (surname) >*
    - Смесен - *<!ELEMENT name (#PCDATA)>*
    - Празен *<!ELEMENT br EMPTY>*
    - Any *<!ELEMENT description ANY>* - всеки елемент, деклриран в DTD може да бъде използван в description в какъвто и е да е ред, колкото и да е пъти.

Всеки елемент, който е посочен в модела на съдържание на елемента, трябва да има своя дефиниция в DTD.
Има 2 начина за определянето на елементите наследници:

- Последователности (sequences)- *<!ELEMENT contact (name, location, phone, knows, description)>* - указва, че елементът contact съдържа елементите name, location, phone, knows, description.
- Избор (choices) - *<!ELEMENT location (address|GPS)>* - указва, че елементът location съдържа елементът address или елементът GPS.

Кардналност на елемент (cardinality) – дефинира колко пъти елементът може да се появи в модела на съдържание. Всеки елемент може да има индикатор колко пъти се среща. Допустимите индикатори са [none], ?, +, *.

Декларация на атрибут

<!ATTLIST contacts source CDATA #IMPLIED>

Декларацията на атрибут се състои от следните 3 части:

- Ключовата дума ATTLIST
- Името на елемента, чийто атрибути се дефинират
- Списък с атрибути – състои се от:
    - Име на атрибута
    - Тип
    - Декларация на стойността

Типовете атрибути са CDATA, ELEMENT, ID, IDREF, IDREFS, ENTITY, ENTITIES, NMTOKEN, NMTOKENS.
Атрибутът може и да се дефинира като списък със допустими стойности.
<!ATTLIST doc lang (en|bg|ge) #IMPLIED>
DTD позволява да се зададе, че атрибутът има:

- Стойност по подразбиране – default - *<!ATTLIST doc lang (en|bg|ge) en >*
- Фиксирана стойност - #FIXED - *<!ATTLIST doc ver CDATA '1.0'>*
- Е задължителен - #REQUIRED *<!ATTLIST person nationality CDATA #REQUIRED>*
- Е опционален - #IMPLIED - *<!ATTLIST doc lang (en|bg|ge) #IMPLIED>*

Декларация на символни данни (character data)

Понякога текстът трябва да съдъжа неинтерпетирани markup символи. Това става със CDATA декларация:
<![CDATA[Press <<<ENTER>>>]]>

Декларация на Entities

XML документите могат да бъдат разпределени в някоко файла. Всяка единица информация се нарича entity, което има идентифициращо го име. Entities се дефинират с entity декларации и се използват с entity reference.
Има 4 вида entities:

- Вградени entities - *&, <, ', "*
- Character entities – подобни са на вградените, но не се декларират в DTD. Могат да се използват в документа без декларации чрз референции например - *©*
- General entities – декларират се в DTD преди да могат да бъдат използвани в документа. Обикновено се използват за създаването на заменим текст - *<!ENTITY source-text "Alabala">* и се използват чрез референция - *&source-text;*
- Parameter entities – не могат да се използват в съдържанието – единствено в DTD. Могат да е използват за създаването на DTD отняколко файла. Декларират се по следния начин - *<!ENTITY DefaultPhoneKind "Home">*. Реферират се чрез *%DefaultPhoneKind*

Условни секции

Състоят се от:

- Include декларации - *<!INCLUDE[…..]]>* - Включва секция DTD
- Ignore декларации - *<!IGNORE[…]]>* - Изключва секция DTD

Декларация на нотация

Описва външно не-XML entity. Може да указва и helper приложение и документация. Например

<!NOTATION name SYSTEM "external_id">
<!ATTLIST element_name attr_name NOTATION default_value>

XML валидация чрез XML Schema

Спецификации

XML Schema Спецификацията се разделя на 3 части:

- Part 0: Primer - съдържа въведение в XML Schema
- Part 1: Structures - дефинира структурата, дефинициите на типове, атрибути, елементи, групи от атрибути (attribute group), модели от групи (model group), нотации и анотации
- Part 2: Data types - определя типовете данни за елементи и атрибути. Дефинира фасети и възможните стойности за типовете.

Типове данни

XML Schema Recommendation дефинира следните видове данни:

- Built-in datatypes
  - примитивни типове данни – string, boolean, float, double, ID и пр.
  - Derived – language, IDREF, long, int, short и др.
- User-defined datatypes

Дефиниране на потребителски типове данни
Става по 3 начина:

- дефиниране от съществуваши типове данни чрез определянето на 1 или повече фасети.
  <datatype name="TelephoneNumber" source="string">
  <length value="8"/>
  <pattern value="\d{3}-\d{4}"/>
  </datatype>
- Derived Types - може да се прилага форма на subclassing на дефинициите на типове. Има 2 начина за това:
  - derive by extension - разширяване на родителския тип с повече елементи –
    <complexType name="Book" source="cat:Publication" derivedBy="extension">

- derive by restriction - ограничаване на родителския тип чрез ограничения върху възможните стойности или броя на occurrences.

  < complexType name= "SingleAuthorPublication" source="cat:Publication" derivedBy="restriction">

- <simpleType> декларации

Фасети

Фасетите контролират всички прости типове в XML Schemas. Фасет е качество или отличителна черта на <simpleType>. Има 12 ограничаващи фасети:

- minExclusive
- minInclusive
- maxExclusive
- maxInclusive
- totalDigits - брой цифри за числов тип
- fractionDigits - брой дробни цифри за числов тип
- length - брой елементи в списък или брой символи на стринг
- minLength
- maxLength
- enumeration
- whiteSpace - определя как ще се третира whitespace в типа
- pattern - позволява рестрикцията на символни типове чрез регулярни изрази
- precision
- scale
- encoding
- duration
- period

Др. вид фасети са фундаменталните (не могат да се променят):

- equal - всички типове осигуряват релация равенство
- order - някой типове осигуряват и order релация
- bounds - горна и долна граница
- cardinality
- numeric

Структури

Структурните типове се използват за описание на елементи, които имат асоциирани към тях елементи наследници или атрибути.

<element> декларации

При деклариране на елемент се задават името му и допустимото му съдържание:

<element
name=”name of the element”
type=”global type”
ref=”global element declaration”
form=”qualified or unqualified”
minOccurs=”non negative number”
maxOccurs=”non negative number or ‘unbounded’”
default=”default value”
fixed=”fixed value”>

Допустимото съдържание се определя от неговия тип. XML Schemas позволяват определянето на типа на елемента по 2 начина:

- създаване на локален тип - нямат <schema> като директен родител и могат да се ползват само в определен контекст.

- използване на глобален тип - глобалните декларации могат да се преизползват в XML Schema чрез добавянето на ref атрибут - <element ref="target:first"/>. Задължително глобалният тип трябва да е именован.

<complexType> декларации

Чрез тях се създават сложни потребителски типове. В рамките на дефиницията може да се укаже допустимото съдържание на елемента.

<complexType mixed="true or false" name="Name of complexType">

<simpleType> декларации

Винаги когато се декларира <simpleType> той трябва да се базира на съществуващ тип данни.

<simpleType name="name of the simpleType" final="#all or list or union or restriction">

```
<simpleType name="sku" base="xsd:string>
    <pattern value="\d{3}-[A-D]{4}"/>
</simpleType>
```

<group> декларации

Дефинират преизползваеми групи от елементи - чрез тях може лесно да се преизползват и комбинират цели модели на съдържание. Дефинициите на глобални групи трябва да са именовани - чрез добавянето на атрибут *name*. Могат да се дефинират групи единствено от елементи.

```
<group name="NameGroup">
        <!-- content model goes here -->
</group>
```

Към дефиницията на групата може да има атрибут order със следните допустими стойности:

- *choice* - инстанциите на групата съдържат някой от елементите
- *all* - инстанциите на групата съдържат всички елементи
- *seq* - - инстанциите на групата съдържат всички елементи в зададения ред

Модели на съдържание (Content Models)

В XML Schemas моделът на съдържание на елементите може да се дефинира чрез:

- <sequence> декларация - указва последователност от елементи, които се съдържат в реда на указването им.
  <sequence minOccurs="non negative number" maxOccurs="non negative number or unbounded">
- А <choice> декларация - могат да се зададат множество декларации наследници. В документа обаче само 1 от декларациите може да бъде използвана.
  <choice minOccurs="non negative number" maxOccurs="non negative number or unbounded">
- Референция към глобална <group> декларация -
  <group ref="global group definition"
  minOccurs="non negative number"
  maxOccurs="non negative number or unbounded">
- An <all> декларация - позволява декларирането на елементи, които могат да се появят в какъвто и да е ред.
  <all minOccurs="0 or 1" maxOccurs="1">
- Вътрешни модели на съдържание
- Element декларация
- Element wildcards
- any Елемент - Позволява всеки well-formed XML. Може да бъде ограничен чрез атрибут Namespace, който да указва от кой namespace са допустимите елементи.
  <xsd:any namespace="##any" minOccurs="0" maxOccurs="unbounded"/>

<attribute> декларации

Винаги са последни след декларациите на всички елементи.

<attribute name="name of the attribute"

type=”global type”
ref=”global attribute declaration”
form=”qualified or unqualified”
use=”optional or prohibited or required”
default=”default value”
fixed=”fixed value”>

Както при декларациите на елементи има 2 метода за деклариране на атрибути:

- създаване на локален тип
- използване на глобален тип

Декларациите на атрибути могат да са единствено прости типове (simple types). Може да се преизползват атрибути чрез реферирането на глобални декларации - <attribute ref=”contacts:kind”/>.

<attributeGroup> декларации

Чрез тях могат да се дефинират преизползваеми множества от атрибути. Глобалните <attributeGroup> декларации трябва да са именовани с name атрибут.
<attributeGroup name=”ContactsAttributes”>
<!-- attribute declarations go here -->
</attributeGroup>

Сравнение с DTD

DTD предоставят фундаменентална граматика за дефиниране на XML документ под формата на метаданни, които описват съдържанието на документа. XML Schema предоставя това плюс детайлен начин за дефиниране на данните, които могат и не могат да се съдържат.
Разликите между XML Schema и DTD са:

- DTD не поддържат namespace за разлика от XML Schema, която пълно поддържа Namespace Recommendation.
- DTD осигуряват възможности за дефиниране на заменим текст, външно съдържание и условна обработка (ENTYTY декларациите). Това не е възможно в XSD.
- DTD описват целия документ, schema-ите могат да дефинират само части от него.
- XSD има дефинирана система от типове и позволява дефинирането на потребителски типове данни, както и наследяване, енкапсулация и субституция.
- XSD е по-богат език за описание на елементи и атрибути. DTD поддържа 10 типа, XSD - 37+.
- DTD може да бъде част от XML документа, докато XSD е задължително в отеделен файл.
- Официалната дефиниция на “валиден XML” изисква DTD.
- XSD може да дефинира много повече ограничения от DTD.
- XSD използва XML синтаксис, DTD има отделен синтаксис. XSD синтаксисът е по-експресивен от този на DTD
- XSD има повече поддържащи средства (DOM, browsers)
- XSD има по-добър модел на съдържанието – напр. Може точно да се задава ограничението за брой срещания на даден елемент
- XSD позволява лесно създаване на сложни и преизползваеми модели на съдържание.
- XSD е съвместима с други XML технологии като Web Services, XQuery, XSLT...
- XSD поддържа DOM и DOM манипулации
- XSD поддържа не само глобални, но и локални имена
- XSD има неограничена възможност за разширяване

Използване на XSLT (eXtensible StyleSheet Language Transformations) и XPath за алокиране, манипулиране и представяне на XML съдържание.

XSLT се използва съвместно с XPath за алокиране, манипулиране и представяне на XML съдържание. XSLT представлява език за трансформация на XML документи в какъвто и да е текстово-базиран формат. Езикът позволява дефинирането на шаблони, коио да се приложат към определена част от XML документ. Към коя част да се приложат се задава чрез XPath изрази.

XSLT спецификацията се разделя на две – XSL Formatting objects (или само XSL) и XSL Transformations. Първото е език за форматиране на XML данни за представянето им на екран, хартия или друг медиен носител. Второто е език за трансформиране на XML документи в други документи.

Асоцииране на XML документ с XSLT Style sheet се извършва чрез инструкция за обработка - *<?xml-stylesheet href="myStyles.xsl" type="text/xsl" ?>*

XPath

XPath – схема за локализиране и идентифициране на подструктури в документи; използва се от други XML технологии като XSLT, XPointer, XQL като средство за локализиране на определени части от XML съдържанието.

XPath модел на данните – при XPath повечето части на един сериализиран XML документ се представят като върхове на дърво. Коренът на дървото е самият XML документ – отбелязва се с '/'. За всеки елемент, атрибут, инструкция към процесора или коментар има съответен връх в дървото. Също така има текстови върхове, представящи текстовото съдържание на елементите.

XPath се възползва от последователния и йерархичен характер на XML документите, за да открие елементи според техния контекст (например къде в йерархията се намира даден елемент)

Контекстен възел – селектира се с .; това е текущата секция в XML документа, от където започва XPath изразът

XPath връх – може да бъде всяка част от XML документ – елемент, атрибут, инструкция към процесора или нещо друго

Корен на документа (document root) – не е първият елемент, а самият XML документ; означава се с '/'

XPath израз – текстов стринг, с който се избира даден елемент, атрибут, инструкция или текст; може да се намира в URL или като стойност на атрибут (http://abc.com/getQuery?/book/intro/title или <xsl:pattern match="chapter/title">…</xsl:pattern>). Изразите локализират нещата по тяхното място в XML йерархията

4 типа изрази – булеви, множества от върхове, числа, стрингове (това са типовете, които могат да бъдат върнати вследствие на оценката на XPath израза)

Location path – повечето XPath изрази връщат множество от върхове, тези изрази се наричат location paths. Пътищата, посочващи местонахождението (location paths), са съставени от стъпки. Всяка стъпка е съставена от ос, връх и предикат (опционален). Когато оста липсва, подразбира се child.

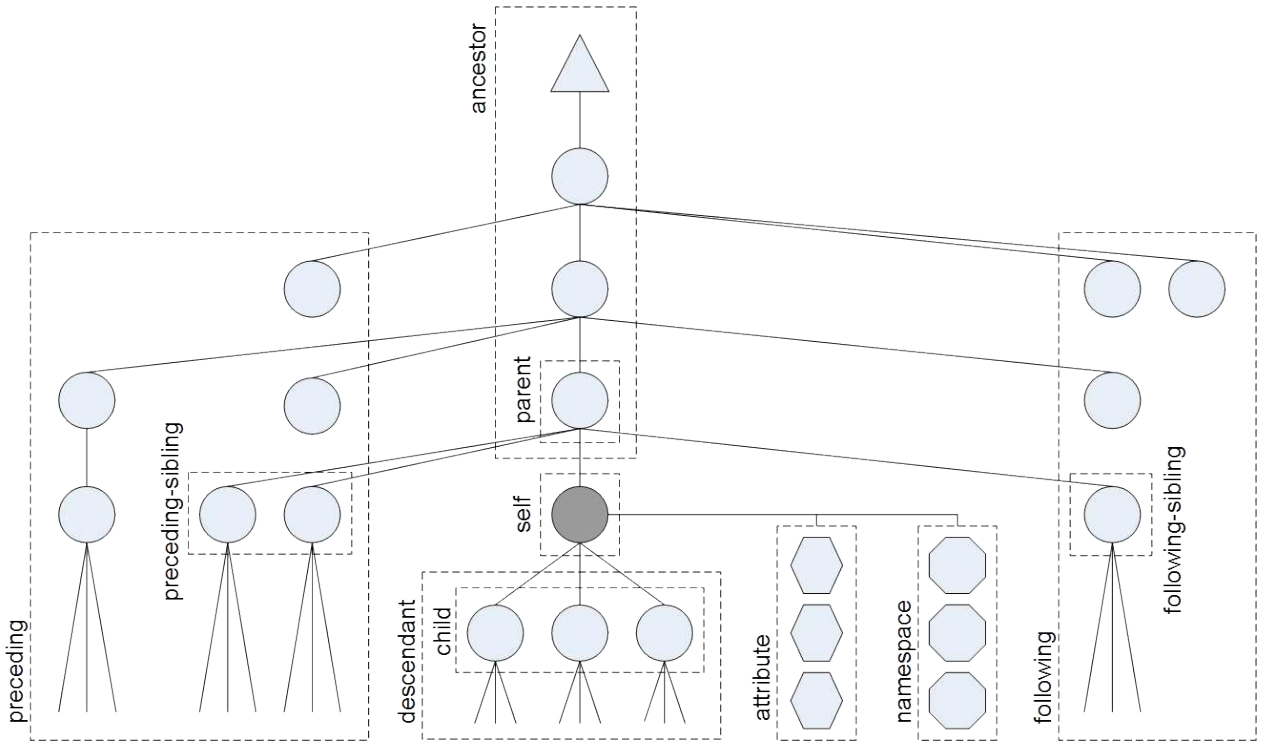
- ✓ Relative paths **–** започват от контекстния връх
- ✓ Absolute paths **–** започват от корена

Видове върхове

- ✓ Корен
- ✓ Възел на елемент
- ✓ Възел на атрибут – възелът на елемента, към който е този атрибут, се нарича възел родител. Всеки възел на атрибут има име и стойност.
- ✓ Текстов възел

- ✓ Възел за пространство от имена
- ✓ Възел за коментар
- ✓ Възел за инструкция към процесора

13 XPath оси – използват се така: име_на_оста::име_на_връх. Оста на контекстния връх се нарича self. Той може да се достъпи по два начина: '.' и self::node.



Заб: На картинката липсват descendant-or-self и ancestor-or-self. С тях не се селектират братя и сестри.

Рекурсивен оператор за достъп до всички потомци – '//'. Пример: chapter//para ще селектира всички para елементи, които са по-ниско в йерархията от chapter. Несъкратеният запис е: child::chapter/descendant-or-self::node()/child::para. Пример: За избиране на всички para елементи в текущия – .//para или descendant-or-self::node()/child::para.

С '..' се селектира родителят на контекстния връх, може да се достъпи и с parent оста.

С '@' се селектира стойността на атрибут

Функции

- ✓ node()
- ✓ text()
- ✓ position()
- ✓ last()
- ✓ count()
- ✓ id()
- ✓ concat()

Предикати – задават се с начупени скоби ([]). Предикат може да се сложи след всяка стъпка в XPath израза, като на една стъпка могат да бъдат зададени и няколко предиката на стъпка.

Манипулиране на XML съдържание

Реализира се чрез дефиниране на правилата за трансформация чрез елемент <xsl:template>. В атрибутът му match се задава XPath израз, който определя върху кои възли ще бъдат приложени шаблоните.

```
<xsl:template match="exon">
        We have found the EXON tag!
</xsl:template>
```

Елементът apply-templates позволява изливкването на шаблони от текущо изпълнявания такъв. Това предизвиква рекурсивната обработка на всички наследници на обработвания елемент.

```
<xsl:template match="p">
        <xsl:apply-templates/>
</xsl:template>
```

Извикване на темплейт се реализира чрез call-template елементът. XSLT позволява и предаването на параметри при извикването - <xsl:with-param name="prefix">new</xsl:with-param> - предава стойност на параметъра при извикването. Дефинирането на параметър става чрез param елемент.

```
<xsl:template match="title">
        <xsl:call-template name="CreateHeader" />
</xsl:template>
```

Изходните елементи могат да бъдат сортирани чрез xsl:sort елемент, който извършва сортировка по определено property.

```
<xsl:template match="list">
        <xsl:apply-templates>
                <xsl:sort select="@code"/>
        </xsl:apply-templates>
</xsl:template>
```

Итерация върху XML съдържание

<xsl:for-each> елементът може да селектира и итерира по всеки XML елемент от определено множество възли.

```
<xsl:for-each select="catalog/cd">
        ......................
</xsl:for-each>
```

Условна обработка на XML съдържание

Има 2 механизма за условна обработка:

- <xsl:if> елемент - съдържа шаблон, който да бъде приложен само ако зададеното условие е истина.

```
<xsl:if test="position() = 1">
        <xsl:attribute name="style">color: red</xsl:attribute>
</xsl:if>
```

- <xsl:choose> елемент заедно с <xsl:when> и <xsl:otherwise> - изразява множество условни тестове.

```
<xsl:choose>
        <xsl:when test="salary[number(.) &gt; 2000]">
                A big number
        </xsl:when>
        <xsl:when test="salary[number(.) &gt; 1000]">
                A medium number
        </xsl:when>
        <xsl:otherwise>A small number
        </xsl:otherwise>
</xsl:choose>
```

Деклариране на променливи

<xsl:variable> - могат да се декларират променливи; за да се достъпи стойност на променлива се ползва '$' (<xsl:value-of select="$colour"/>); също така стойността на променливата може да се ползва и в елементи от резултатното дърво (<ajr:glyph colour="{$colour}"/>). По този начин могат да се добавят прости константи (например да си дефинираме числото пи).

Алокиране на XML съдържание

XSLT позволява и създаването на елемент, атрибути и групи от атрибути в изходния документ. Това се реализира чрез xsl:element, xsl:attribute и xsl:attribute-set. Алокирането на динамични елементи и атрибути може да се реализира чрез задаването на XPath изрази за стойност или име.

```
<xsl:element namespace="html" name="lala">
<xsl:attribute name="style">purple</xsl:attribute>
```

Копиране на елементи

Има 2 възможности за копиране:

- Плитко – реализира се чрез copy елемент, който копира контекстния възел без наследниците и атрибутите му.

```
<xsl:template match="h1|h2|h3|h4|h5|h6|h7">
    <xsl:copy>
            Header: <xsl:apply-templates/>
    </xsl:copy>
</xsl:template>
```

- Дълбоко – елементът copy-of копира контекстния възел заедно с наследниците и атрибутите му.

```
<xsl:copy-of select="//h1 | //h2" />
```

Извличането на информация от XML дървото се реализира чрез елемента <xsl:value-of>. Той взима информация от обработвания XML и я добавя към резултатното дърво. В атрибута select се указва чрез XPath израз какво да бъде селектирано, като се включват всички наследници. Този елемент свежда селектирания обек до символен низ.

```
<xsl:template match="b">
```

```
        <xsl:value-of select="." />
</xsl:template>
```

Представяне на XML съдържание

Тъй като XML данните обикновено не съдържат елементи, които определят как да се презентират данние, е необходимо да се използват други методи за представяне. Един от тях е XSL документи, трансформиращи XML. Представянето на данните се реализира чрез асоцииране на XSL документ към XML документа и обработката на XML съдържанието от XSLT процесора по зададените критерии.

Контрол върху изходния документ се получава чрез <xsl:output>, който позволява задаването на формата, версията, енкодинга и др.

Използване на DOM (Document Object Model) и SAX (Simple API for XML) за обработка на XML документи

Основни интерфейси на DOM и SAX и начини за използването им

Има три основни стъпки при използването на XML парсер:

1. Създаване на обект от типа на парсера
2. Подаване на XML документа към парсера
3. Обработка на резултатите

В общия случай генерирането на XML съдържание е извън възможностите на парсерите, но има и такива, които могат да поддържат и такава функционалност.

Двата главни програмни интерфейса (APIs) за парсване на XML са SAX и DOM.

DOM интерфейси

DOM дефинира няколко следните Java интерфейси:

- DOMImplementation - Интерфейс за създаване на началния Document възел на DOM дървото.
- Node Interface - Представя основния обект в DOM - единичен възел в DOM дървото. Той е базовият интерфейс за всички други в DOM и осигурява достъп до общи пропъртита на възли, методи за обхождане и модификация на дървото.
- Document::Node Interface - Представлява целия XML документ (корена на дървото). Освен това се използва за factory за други типове възли - елементи, текстови възли, коментари, CDATA секции, processing инструкции, атрибути, entity референции. Има методи за достъп до DOCTYPE, до информация за възможностите на DOM имплементацията, до корена на дървото и пр.
- DocumentType::Node interface - Предоставя достъп до Entity и Notation колекциите в документа, както и до някой аспекти на външните и вътрешни DTD.
- Element::Node Interface - Съдържа 2 категории методи:
  - общи методи за елементи - за получаване на името на таг, на елементите по таг, нормализиране
  - методи за манипулиране на атрибути - за взимане и промяна на атрибут, премахване на такъв и пр.
- Attr::Node Interface - моделира атрибутите в XML документ, осигурявайки достъп до различни properties на атрибута. Поддържа методи за достъп до име и стойност и промяната им. Въпреки че extend-ва Node интерфейсът, атрибутите не се считат за част от DOM дървото.
- Entity::Node Interface - представя вътрешно или външно entity в XML документ.

- DocumentFragment::Node Interface - е signature interface. Той съхранява части от Document, когато се добави към друг Node се премахва. Не е нобходимо да се съобразява с правилата за XML.

SAX Интерфейси

SAX API са разделени на следните 4 области:

- core interfaces - облекчават работата с core информацията в XML документ
- core classes - облекчават работата с core информацията в XML документ
- extended interfaces - моделират аспекти на документа, които не представляват интерес за повечето програмисти - DTD декларации, коментари и пр.
- helper classes - съдържат няколко класа за удобство, както и имплементациите по подразбиране на някои от core интерфейсите.

Attributes - моделира атрибутите на елементи. Те се предоставят чрез ненаредено множество от свойства, което може да бъде обхождано по име и позиция.

ContentHandler - основен SAX интерфейс. Той моделира core информацията за XML документ като подредена последователност от извиквания на методи и предоставя интерфейс за получаване на основни markup събития от парсера.

DTDHandler - предоставя callback методи за получаване на нотификации за DTD събития.

EntityResolver - интерфейс, позволяващ на имплементациите да осигуряват custom резолюция на външни entities.

ErrorHandler - моделира well-formed грешки, грешки при валидация и warrinig-и. Консуматорът на ContentHandler имплементацията използва този интерфейс да преустанови потокът от извиквания на методи, които довеждат до грешки. ErrorHandler дефинира 3 нива exceptions - error, fatalError, warning.

Locator - парсерът може да предостави Locator обект, който да бъде използван в event методите, за да се разбере къде точно в документа е текущия метод.

XMLFilter - подинтерфейс на XMLReader. Повечето SAX интерфейси позволяват pipeline обработка, където имплементацията на ContentHandler може да получава някой информационни елементи, които разпознава, но да предава неразпознатите елементи на следващият процесор във веригата. Това се реализира чрез XMLFilter интерфейс. Той работи върху събия вместо върху документа и се регистрира при предишния филтър

XMLReader - представлява интерфейс за четене на XML документ чрез използването на callback механизъм. Позволява на приложенията да задават и получават свойства в парсера, да се регистрират event handler-и за обработка на документ и да се инициира парсването на документа.

Сравнение между DOM и SAX

DOM създава дърво за целия документ в паметта, докато SAX парсва документа възел по възел. DOM е подходящ, в случай че е необходимо да се обработва голяма част от елементите в документа и да се извършват операции върху XML. Подходящ е и при необхдимост за множество достъпи до XML. SAX е подходящ при големи XML документи, в които за обработката са необходими малко елементи, идеален е за извършването на прости операции върху XML файлове. SAX е разработен с цел да предостави по-ефективен анализ на големи документи. Той е подходящ за обработка по време на парсването на документа, тъй като единствено последния event е в паметта. Обработката със SAX изисква по-малко място и време, тъй като не се построява карта на целия документ в паметта, и е подходящ за сложни структури. Тъй като SAX е създаден единствено за четене на XML, но не и за писане при необходимост от това се използва DOM.
DOM е подходящ за приложения, който изискват структурни манипулации върху много XML tokens, а SAX - за приложения с ограничения в паметта.